Type of Project: Application

Project Overview:

Crowdfunding has become a revolutionary method for raising capital through the collective effort of individuals, primarily via online platforms. However, traditional crowdfunding systems often face critical issues related to security, transparency, trust, and efficiency. These problems limit the potential of such platforms, particularly when it comes to protecting backers' funds and ensuring that campaign promises are fulfilled. This project, titled "Crowdfunding Using Blockchain," presents a solution to these challenges by integrating blockchain technology into the crowdfunding process. Blockchain offers a decentralized, tamper-proof ledger that records all transactions transparently and securely. It eliminates the need for intermediaries and introduces smart contracts—self-executing agreements that ensure funds are automatically released only when predefined conditions are met.

The project demonstrates how blockchain can:

- Enhance security by preventing data tampering and fraud.
- Improve transparency by making all campaign transactions visible and verifiable on a public ledger.
- Increase trust between fundraisers and backers by automating fund disbursement through smart contracts.
- Allow global participation through the use of cryptocurrency payment gateways, breaking down traditional financial barriers.

The system architecture includes multiple modules, such as blockchain integration, smart contract automation, cryptocurrency handling, and a user-friendly interface. The platform supports three user types—admins, fundraisers, and backers—and provides a seamless experience for creating, managing, and backing campaigns.

By combining blockchain’s core principles—decentralization, immutability, and programmability—this project aims to revolutionize the way crowdfunding platforms operate, making them more secure, accountable, efficient, and globally accessible.

ABSTRACT

Crowdfunding has emerged as a popular method for raising funds from a large pool of individuals through online platforms. However, traditional crowdfunding models often face challenges related to security, transparency, and efficiency. The integration of blockchain technology has revolutionized this process by offering enhanced security, transparency, and efficiency. Blockchain ensures that all transactions and data associated with the crowdfunding campaign are securely stored in a decentralized manner, mitigating the risks of fraud or manipulation. Smart contracts, embedded within the blockchain, facilitate automatic execution of agreements between fundraisers and backers, thereby ensuring trust and accountability throughout the crowdfunding process. This paper explores the transformative impact of blockchain on crowdfunding, highlighting its potential to reshape the landscape of fundraising by providing a more secure, transparent, and efficient platform for both fundraisers and backers.

LIST OF CONTENTS

LIST OF SYMBOLS & ABBREVIATIONS

ABBREVIATIONS	EXPANSION
1.RF	Random Forests
2.KNN	K-Nearest Neighbor
3.SVM	Support Vector Machine
4.NPM	Node Package Manager
5.DAO	Decentralized Autonomous Organization
6.DeFi	Decentralized Finance
7.CLI	Command Line Interface
8.SPA	Single Page Application
9.IoT	Internet of Things
10.API	Application Programming Interface
11.JSON	JavaScript Object Notation
12.CRUD	Create, Read, Update, Delete
13.ORM	Object-Relational Mapping
14.CMS	Content Management System
15.ES6	ECMAScript 6 (a version of JavaScript)
16.V8	Google's open-source JavaScript engine
17.HTTP	Hypertext Transfer Protocol
18.HTTPS	Hypertext Transfer Protocol Secure
19.REST	Representational State Transfer
20.GraphQL	Graph Query Language

CHAPTER 1

1.INTRODUCTION

1.1 OVERVIEW

In recent years, crowdfunding has emerged as a powerful tool for individuals and organizations to raise funds for various projects, ventures, and causes. This practice involves soliciting small contributions from a large number of people, typically via online platforms, to collectively fund a specific initiative. While crowdfunding has democratized access to capital and empowered innovators and entrepreneurs, it has also faced challenges related to security, transparency, and efficiency. Traditional crowdfunding platforms often rely on centralized systems to manage transactions and data, which can be vulnerable to security breaches, fraud, and manipulation. Moreover, the lack of transparency in the allocation and use of funds can erode trust among backers, leading to skepticism and reluctance to participate in future campaigns. Additionally, the manual execution of agreements and the cumbersome administrative processes involved in traditional crowdfunding can result in delays, inefficiencies, and disputes between fundraisers and backers. The integration of blockchain technology promises to address these challenges and revolutionize the crowdfunding landscape. Blockchain, a decentralized and immutable ledger system, offers enhanced security by cryptographically securing transactions and storing data across a distributed network of nodes. This decentralized nature of blockchain reduces the risk of fraud or manipulation, as transactions are transparent and tamper-proof, providing stakeholders with greater confidence in the integrity of the crowdfunding process. Furthermore, smart contracts, self-executing agreements programmed on the blockchain, automate the execution of terms and conditions agreed upon by fundraisers and backers. Smart contracts facilitate trust and accountability by ensuring that funds are released to fundraisers only when predetermined conditions are met, such as reaching fundraising goals or completing project milestones. This automated and transparent approach streamlines the crowdfunding process, reduces administrative overhead, and minimizes the potential for disputes or conflicts between parties. Overall, the integration of blockchain technology offers a paradigm shift in crowdfunding, providing a more secure, transparent, and efficient platform for both fundraisers and backers. By leveraging blockchain's inherent features of decentralization, immutability, and programmability,

crowdfunding campaigns can foster greater trust, participation, and success, unlocking new opportunities for innovation, collaboration, and social impact.

1.2 OBJECTIVE

The objective of this project is to explore and demonstrate the transformative potential of integrating blockchain technology into the crowdfunding ecosystem. By leveraging the unique features of blockchain, such as decentralization, transparency, and programmability, the project aims to address key challenges faced by traditional crowdfunding models, including security vulnerabilities, lack of transparency, and inefficiencies in the execution of agreements. Through research, analysis, and practical implementation, the project seeks to showcase how blockchain can enhance the security of crowdfunding transactions, mitigate the risk of fraud or manipulation, and provide stakeholders with greater transparency into the allocation and use of funds. Additionally, the project aims to highlight the role of smart contracts in automating the execution of agreements between fundraisers and backers, thereby fostering trust, accountability, and efficiency throughout the crowdfunding process. By demonstrating the benefits of blockchain-enabled crowdfunding, the project intends to contribute to the broader understanding of how emerging technologies can reshape fundraising practices, empower entrepreneurs and innovators, and facilitate greater collaboration and participation in crowdfunding campaigns. Ultimately, the objective is to provide insights and recommendations for leveraging blockchain to create a more secure, transparent, and efficient crowdfunding ecosystem that benefits both fundraisers and backers alike.

CHAPTER 2

LITERATURE SURVEY

2.1 TITLE: Blockchain-Based Crowdfunding Application

AUTHOR: V. Patil, Vasvi Gupta, Rohini Sarode

YEAR: 2021

DESCRIPTION

People's data is valuable and sensitive, and blockchain can significantly change how it is seen. All transactions are time- and date-stamped and are logged irreversibly. Smart contracts can even automate transactions, boosting your productivity and speeding up the process even further. After pre-specified conditions are met, the transaction or process moves on to the next stage. Smart contracts eliminate the need for human intervention and the reliance on third parties to verify that contract requirements have been satisfied. The issue of transparency and security is very paramount in any organization, especially in organizations providing crowdfunding platforms, therefore the intention to provide a reliable, secured, transparent and decentralized solution is achieved by developing a blockchain-based crowdfunding web application. This crowdfunding application is not just like any other application which just allows people to invest their money, but this platform also gives an assurance to the backers that returns will be guaranteed. The application will also provide transparency between the backers and the start-ups so that the backers can stay updated on the progress of the project work of the respective start-ups that they invested their money in. Money will be refunded to the backers in case the project is aborted in between.This will be a multi-user application with three different types of users: Admin, Backers, and Start-up. Admin can approve start-ups for listing.

2.2 TITLE: Medtrust - A Decentralized Model for Medical Crowdfunding Based on Smart Contract Using Ethreum

AUTHOR: Dr Nirmal K Raja, Dr Prem

Sankar YEAR: 2023

DESCRIPTION

Trust and Transparency is a great challenge that the current world encounter when coming with systems that really matters the day to day life. When it comes to financial systems, the requirement becomes stronger. The paper suggests the implementation of a medical crowdfunding system incorporating trust and transparency with blockchain. With decentralized approach. The paper explains implementation of a Crowdfunding system based on blockchain using smart contract. The Paper explains the architecture, implementation and Test cases that been used and a study of the how the model solves the issue of trust. The system is been tested and deployed with hardhat framework over sepolia Test Network.

2.3 TITLE: Design and Development of Ethereum based Crowdfunding using Blockchain Technology

AUTHOR: Kaustubh Anavkar, Ashish Vishwakarma

YEAR: 2023

DESCRIPTION

A growing number of enterprises, including start-up firms, artistic activities, and charitable causes, are being funded through crowdfunding. Traditional crowdfunding websites, on the other hand, struggle with a variety of issues, including prohibitive pricing, a lack of transparency, and fraud concerns. These issues have led to an increase in the use of blockchain technology as a crowdfunding platform. A decentralized, open-source ledger using blockchain technology safely and permanently records transactions. It provides several advantages for crowdsourcing, including decreased costs, increased transparency, and improved efficiency. Blockchain technology has the potential to open up crowdfunding to a wider range of investors, particularly those in developing countries with limited access to traditional financial institutions. Blockchain technology can be utilised for crowdfunding and smart contracts as well. Numerous procedures associated with crowdfunding, such as fund disbursement and investor verification, can be automated thanks to these self-executing contracts. Automation allows for a significant reduction in administrative work and a faster, more precise crowdfunding procedure. The goal of this article is to advance understanding regarding the application of crowdfunding using blockchain technology. We'll review the body of literature on blockchain and crowdfunding, take into account the benefits and drawbacks of doing so, and offer a framework for implementing blockchain-based crowdfunding. Our study will contribute to the growing body of knowledge on blockchain and crowdfunding, which will be helpful to policymakers, company owners, and investors interested in using blockchain for crowdfunding.

2.4 TITLE: Blockchain Technology: Benefits, Challenges, Applications, and Integration of Blockchain Technology with Cloud Computing

AUTHOR: Gousia Habib, Sparsh Sharma

YEAR: 2022

DESCRIPTION

The real-world use cases of blockchain technology, such as faster cross-border payments, identity management, smart contracts, cryptocurrencies, and supply chain–blockchain technology are here to stay and have become the next innovation, just like the Internet. There have been attempts to formulate digital money, but they have not been successful due to security and trust issues. However, blockchain needs no central authority, and its operations are controlled by the people who use it. Furthermore, it cannot be altered or forged, resulting in massive market hype and demand. Blockchain has moved past cryptocurrency and discovered implementations in other real-life applications; this is where we can expect blockchain technology to be simplified and not remain a complex concept. Blockchain technology's desirable characteristics are decentralization, integrity, immutability, verification, fault tolerance, anonymity, audibility, and transparency. We first conduct a thorough analysis of blockchain technology in this paper, paying particular attention to its evolution, applications and benefits, the specifics of cryptography in terms of public key cryptography, and the challenges of blockchain in distributed transaction ledgers, as well as the extensive list of blockchain applications in the financial transaction system. This paper presents a detailed review of blockchain technology, the critical challenges faced, and its applications in different fields.

2.5 TITLE: Revolutionizing Crowdfunding Using Blockchain Technology
AUTHOR: Himanshu Mehra, Chirag Bansal, Jatin Chopra
YEAR: 2023

DESCRIPTION

Cryptocurrencies are a game-changer in the world of finance and crowdfunding. They offer a secure and decentralized way for businesses to generate funds. Unlike traditional crowdfunding websites, cryptocurrency systems provide transparency and control over donated funds. Social media and crowdfunding platforms make it easier than ever to attract investors and entrepreneurs. Cryptocurrencies can give entrepreneurs access to international capital, improve transparency, and lower transaction costs. However, it is important to be aware of the risks, such as market volatility, security concerns, and regulatory issues. It is an exciting field with lots of potential. Cryptocurrency, like Bitcoin, has gained popularity as a decentralised digital currency, and crowdfunding has become a popular way for startups and projects to raise funds. High-level languages such as Solidity provide an abstract and user-friendly approach to developing smart contracts for crowdfunding platforms. These languages offer a more accessible way for developers to create and manage transactions, ensuring transparency and security.

CHAPTER 3

3.1 EXISTING SYSTEM

In the existing system is crowdfunding is important for backing innovative projects and new startup businesses. However, success in achieving the target fundraising is a big challenge, and it depends on many complex factors. In the existing method algorithm is three machine learning models were constructed in this study using: (1) Random Forests (RF), (3) K-Nearest Neighbor (KNN), and Support Vector Machine (SVM) algorithms. The models were trained using a separate portion representing two-thirds of the dataset, while the remaining third was used for evaluation. ChatGPT In the existing crowdfunding system, supporting innovative projects and new startup businesses is crucial, yet achieving fundraising targets poses a significant challenge influenced by numerous complex factors. To address this challenge, researchers have employed machine learning techniques to develop predictive models aimed at improving the success rate of crowdfunding campaigns. In a recent study, three machine learning models were constructed using Random Forests (RF), K-Nearest Neighbor (KNN), and Support Vector Machine (SVM) algorithms. These models were trained using a subset representing two-thirds of the dataset, while the remaining third was utilized for evaluation purposes. By leveraging these machine learning algorithms, researchers aimed to predict the success of crowdfunding campaigns based on various features and parameters associated with the projects, backers, and campaign dynamics. Random Forests, a versatile ensemble learning method, were utilized to build a predictive model capable of handling complex interactions between features and producing robust predictions. K-Nearest Neighbor algorithm, on the other hand, relied on proximity-based classification, where the success of a campaign was predicted based on the similarity to other successful campaigns in the dataset. Support Vector Machine, known for its effectiveness in handling high-dimensional data, was employed to delineate a hyperplane that separates successful from unsuccessful crowdfunding campaigns in the feature space. By training and evaluating these machine learning models on a comprehensive dataset encompassing various crowdfunding campaigns, researchers aimed to identify key predictors and factors influencing the success of fundraising efforts. Insights derived from these models could inform fundraisers and platform operators about effective strategies, campaign dynamics, and factors driving successful crowdfunding outcomes. Moreover, the predictive capabilities of these models could potentially aid in optimizing campaign strategies, allocating resources more efficiently, and enhancing the overall success rate of crowdfunding

initiatives. Overall, the integration of machine learning techniques into the existing crowdfunding framework represents a promising approach to addressing the challenges associated with achieving fundraising targets. By leveraging data-driven insights and predictive modeling, stakeholders in the crowdfunding ecosystem can make informed decisions, mitigate risks, and increase the likelihood of success for innovative projects and startup businesses.

3.1.1 DISADVANTAGES

- The models can overfit to the training data, leading to poor generalization on unseen data.
- Models trained on one dataset may not generalize well to different crowdfunding platforms or industries, limiting their applicability in diverse contexts.
- Training and evaluating machine learning models can be computationally expensive, especially with large datasets, which may pose scalability challenges for real-time prediction in crowdfunding platforms.

3.2 PROPOSED SYSTEM

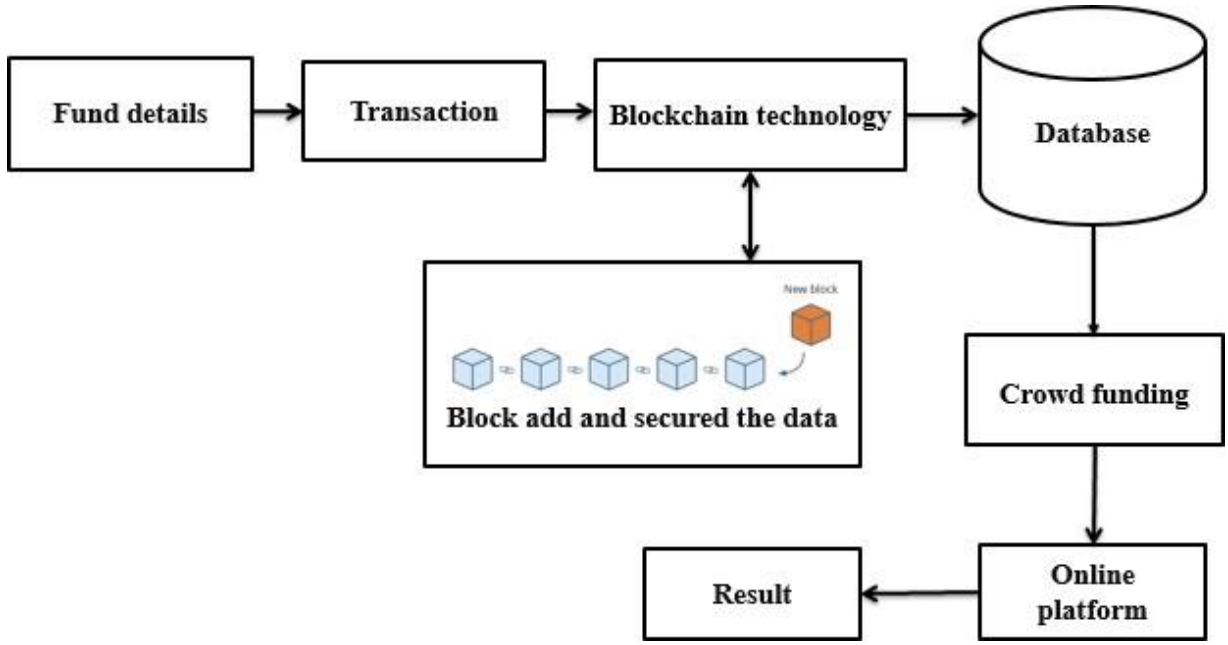
The proposed system aims to leverage blockchain technology to transform existing crowdfunding platforms, offering a more secure, transparent, and efficient fundraising experience. By integrating blockchain's decentralized ledger and smart contract capabilities, the system enhances security, transparency, and efficiency throughout the crowdfunding process. Smart contracts, embedded within the blockchain, automate agreements between fundraisers and backers, ensuring trust and accountability by executing predefined conditions autonomously. This eliminates the need for intermediaries and reduces the risk of disputes or delays in fund disbursement. Additionally, the immutable nature of blockchain records provides a tamper-proof and transparent record of all transactions, enhancing trust and transparency among stakeholders. Furthermore, the use of cryptocurrency enables seamless cross-border transactions, overcoming the limitations and barriers associated with traditional banking systems. This facilitates greater accessibility and inclusivity, allowing individuals from diverse geographical locations to participate in crowdfunding campaigns without the constraints of currency conversions or banking regulations. Overall, the proposed system offers a robust and innovative solution to the challenges faced by current crowdfunding platforms. By harnessing the power of blockchain technology, the system

not only addresses security and transparency concerns but also fosters greater participation and success in fundraising endeavors. This transformative approach has the potential to revolutionize the crowdfunding landscape, unlocking new opportunities for innovation, collaboration, and social impact on a global scale.

3.2.1 ADVANTAGES

- The utilizing blockchain technology ensures data integrity and protection against fraud.
- In improved security and transparency, participants have greater confidence in the crowdfunding process.

3.3 SYSTEM ARCHITECTURE



Fund Details: At the core of the system are the fund details, which include information about the crowdfunding campaigns, such as fundraising goals, project descriptions, and milestones. These details are securely stored on the blockchain, ensuring immutability and transparency. Transactions: Each transaction within the crowdfunding platform, whether it involves contributions from backers or disbursements to fundraisers, is recorded as a transaction on the

blockchain. These transactions are transparent, traceable, and tamper-proof, providing a reliable audit trail of all financial activities. Blockchain Technology: The blockchain technology serves as the underlying infrastructure for the crowdfunding platform. It consists of a decentralized network of nodes, each maintaining a copy of the ledger containing all transactions and fund details. Through consensus mechanisms like Proof of Work or Proof of Stake, the integrity and security of the blockchain are maintained. Database: While the blockchain serves as the primary database for storing fund details and transactions, additional off-chain databases may be utilized for storing supplementary information, such as user profiles, campaign updates, and multimedia content. These off-chain databases interface with the blockchain through secure APIs or protocols. Crowdfunding: The crowdfunding functionality enables fundraisers to create campaigns, set fundraising goals, and attract backers to contribute funds. Backers can browse through available campaigns, review project details, and make contributions using cryptocurrencies or traditional payment methods. Smart contracts govern the execution of agreements between fundraisers and backers, automating fund disbursements based on predefined conditions. Online Platform: The online platform serves as the user interface for interacting with the crowdfunding system. It provides a user-friendly interface for fundraisers to create and manage campaigns, for backers to discover and contribute to projects, and for administrators to oversee platform operations. The platform integrates with the blockchain and off-chain databases to fetch and display relevant information in real-time. Result and Block Addition: As crowdfunding campaigns progress and reach their fundraising goals, the results are recorded on the blockchain as transactions. Each successful fundraising milestone is added as a new block to the blockchain, further securing the data and ensuring its permanence. The addition of blocks follows the consensus rules of the blockchain network, guaranteeing the integrity and validity of the added data

CHAPTER 4

4.1 MODULE LIST

1. Blockchain Integration Module
2. Smart Contract Automation Module
3. Crypto currency Payment Gateway Module
4. Security and Transparency Module
5. User Interface Module

4.2 MODULE DESCRIPTION

BLOCKCHAIN INTEGRATION MODULE: This module is responsible for integrating blockchain technology into the existing crowdfunding platform. It facilitates the creation of decentralized ledgers to securely store fund details and record transactions. Additionally, it manages the deployment and execution of smart contracts to automate agreements between fundraisers and backers.

SMART CONTRACT AUTOMATION MODULE: This module focuses on implementing smart contracts to automate agreements between fundraisers and backers. It defines the conditions under which funds are disbursed, ensuring trust and accountability throughout the crowdfunding process. Smart contracts are programmed to execute predefined actions autonomously based on specific triggers or criteria.

CRYPTOCURRENCY PAYMENT GATEWAY MODULE: This module enables the use of cryptocurrencies for seamless cross-border transactions within the crowdfunding platform. It integrates with cryptocurrency wallets and exchanges to facilitate the acceptance and processing of digital currency payments from backers. By leveraging cryptocurrencies, this module eliminates the barriers associated with traditional banking systems, such as currency conversion fees and transaction delays.

SECURITY AND TRANSPARENCY MODULE: This module focuses on enhancing the security and transparency of the crowdfunding platform through the use of blockchain technology. It ensures that all transactions and fund details are securely stored on the blockchain's decentralized ledger, providing a tamper-proof record of all activities. Additionally, it implements encryption and authentication mechanisms to safeguard sensitive data and protect user privacy.

USER INTERFACE MODULE: This module provides the user interface for interacting with the crowdfunding platform. It offers a user-friendly dashboard for fundraisers to create and manage campaigns, for backers to discover and contribute to projects, and for administrators to oversee platform operations. The user interface integrates with other modules, such as the blockchain integration module and cryptocurrency payment gateway module, to provide a seamless and intuitive user experience.

CHAPTER 5

SYSTEM REQUIREMENTS

5.1 HARDWARE SYSTEM CONFIGURATION:-

- Processor - Pentium – IV
- RAM - 4 GB (min)
- Hard Disk - 20 GB

5.2 SOFTWARE SYSTEM CONFIGURATION:-

- Operating System: Windows 7 or 8
- Frontend : HTML, CSS, REACT
- Backend : Node.js

5.3.1 INTRODUCTION

Node.js is an open-source and cross-platform JavaScript runtime environment. It is a popular tool for almost any kind of project!

Node.js runs the V8 JavaScript engine, the core of Google Chrome, outside of the browser. This allows Node.js to be very performant.

A Node.js app runs in a single process, without creating a new thread for every request. Node.js provides a set of asynchronous I/O primitives in its standard library that prevent JavaScript code from blocking and generally, libraries in Node.js are written using non-blocking paradigms, making blocking behavior the exception rather than the norm.

When Node.js performs an I/O operation, like reading from the network, accessing a database or the filesystem, instead of blocking the thread and wasting CPU cycles waiting, Node.js will resume the operations when the response comes back.

This allows Node.js to handle thousands of concurrent connections with a single server without introducing the burden of managing thread concurrency, which could be a significant source of bugs.

Node.js has a unique advantage because millions of frontend developers that write JavaScript for the browser are now able to write the server-side code in addition to the client-side code without the need to learn a completely different language.

In Node.js the new ECMAScript standards can be used without problems, as you don't have to wait for all your users to update their browsers - you are in charge of deciding which ECMAScript version to use by changing the Node.js version, and you can also enable specific experimental features by running Node.js with flags.

Why Choose Node.js? Key Reasons to Use It in Your Project

Node.js has become one of the most popular backend technologies due to its speed, scalability, and JavaScript-based ecosystem. Here's why you should consider using Node.js for your project:

1. JavaScript Everywhere (Full-Stack Development)

- Use the same language (JavaScript/TypeScript) for both frontend (React, Angular, Vue) and backend (Node.js).
- Reduces context switching and improves team efficiency.

2. High Performance & Scalability

- Powered by Google's V8 engine, which compiles JavaScript to native machine code for fast execution.

- Event-driven, non-blocking I/O allows handling thousands of concurrent connections efficiently (unlike traditional multi-threaded servers like Apache).
- Used by Netflix, PayPal, LinkedIn, and Uber for high-traffic applications.

3. Fast Development with NPM (Largest Package Ecosystem)

- npm (Node Package Manager) has over 2 million packages (libraries & tools).
- Quickly integrate features like:
    - Express.js (for APIs)
    - Socket.IO (real-time apps)
    - Axios (HTTP requests)
    - TypeORM/Prisma (database ORM)

4. Real-Time Capabilities (WebSockets, Streaming, IoT)

- Perfect for:
    - Chat apps (WhatsApp, Slack-like systems)
    - Live updates (stock market dashboards, sports scores)
    - Collaboration tools (Google Docs, Trello)
    - IoT & Gaming (real-time sensor data)

5. Microservices & Cloud-Native Friendly

- Lightweight and modular, making it ideal for:
    - Microservices architecture
    - Serverless functions (AWS Lambda, Vercel, Firebase)
    - Containerized apps (Docker, Kubernetes)

6. Cross-Platform & Easy to Deploy

- Runs on Windows, Linux, and macOS.

- Easy deployment on Vercel, Heroku, AWS, Azure, and more.

7. Strong Community & Corporate Support

- Backed by OpenJS Foundation (Linux Foundation).
- Used by Netflix, Walmart, NASA, and PayPal.
- Large developer community (Stack Overflow, GitHub, Discord).

Uses of Node.js

Node.js is widely used in modern application development due to its speed, scalability, and ability to handle real-time operations. It's suitable for a broad range of applications — from simple websites to complex enterprise-level systems.

Here are the most common and practical uses of Node.js:

1. Web Application Development

Node.js is perfect for building fast, scalable, and lightweight web apps. It can serve both server-side logic and APIs efficiently.

- Dynamic websites
- Admin panels
- E-commerce platforms
- CMS (Content Management Systems)

2. Real-Time Applications

Thanks to its event-driven architecture, Node.js is ideal for apps where real-time data flow is essential.

- Chat applications (e.g., WhatsApp clones)
- Online multiplayer games
- Live support systems
- Real-time dashboards and notifications

3. RESTful APIs and Backend Services

Node.js is often used to create RESTful or GraphQL APIs that serve as the backend for mobile or web applications.

- JSON APIs for mobile apps
- CRUD operations
- Authentication and user management
- Middleware services

4. Single Page Applications (SPAs)

SPAs require fast backends to load data dynamically without refreshing the entire page. Node.js works well with frameworks like React, Angular, or Vue.js.

- Social media platforms
- Portfolio sites
- Blogging platforms

5. Streaming Applications

Node.js handles data streams efficiently. It's a good choice for applications that need to stream media or process large files.

- Video/audio streaming apps (like Netflix)
- Podcast platforms
- Upload/download managers

6. IoT (Internet of Things)

Node.js is lightweight and works well with data-intensive applications that run across distributed devices.

- Smart home systems
- Sensor networks
- Remote device monitoring

7. Microservices Architecture

Node.js fits perfectly into a microservices-based architecture because of its speed, modularity, and ability to handle high concurrency.

- Scalable enterprise apps
- Containerized services with Docker or Kubernetes

8. Serverless Applications

With cloud providers like AWS (Lambda), Google Cloud Functions, and Azure Functions supporting Node.js, it's widely used for serverless backend development.

- Event-driven apps
- Lightweight APIs
- Scheduled tasks

9. Command-line Tools

You can use Node.js to create powerful and interactive CLI tools for automation, configuration, or development workflows.

- File converters
- Deployment scripts
- Build tools (Webpack, ESLint, etc.)

10. Cross-platform Desktop Applications

With frameworks like Electron, Node.js allows building desktop apps using web technologies (HTML, CSS, JS).

- Visual Studio Code (built with Electron + Node.js)
- Slack desktop app
- Music and media players

Working with a Node.js Library

1. Introduction to Node.js Libraries

- Node.js libraries are reusable sets of code packaged as modules.
- They help developers avoid writing repetitive or complex code from scratch.
- Libraries are installed using Node Package Manager (NPM).
- Common libraries include express, mongoose, axios, dotenv, and many more.

2. Project Initialization

- Create a new Node.js project:
    - Use npm init or npm init -y to generate package.json.

This file manages project metadata and dependencies.

3. Installing a Library

- Libraries are installed using the NPM command:
    - Example: npm install express
    - This installs the library and adds it to the dependencies section of package.json.

4. Importing the Library

- Libraries are imported using the require() function:
- Example: const express = require('express');
- In ES module syntax (optional): import express from 'express';

5. Using the Library

- After importing, you can call methods or classes provided by the library.
- Example using **Express.js**:
    - Create a web server
    - Define routes
    - Handle HTTP requests and responses

```
javascript
CopyEdit
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello World');
});

app.listen(3000);
```

6. Running the Application

- Use the command: node app.js (or whatever the main file is)
- This executes your Node.js program with the integrated library

7. Library Functionality

- Libraries provide modular functionality like:
    - Web frameworks (express)
    - Database handling (mongoose, mysql)
    - Authentication (jsonwebtoken, bcrypt)
    - Networking/API (axios, request)
    - Environment configuration (dotenv)

8. How Libraries Work in Node.js

- Node.js searches for the module in the node_modules directory.
- Each library is a folder with a package.json and entry point (e.g., index.js).
- You interact with the library through its **exported APIs**.

9. Benefits of Using Libraries

- Saves development time
- Promotes modular, clean code
- Leverages community-tested and optimized solutions
- Keeps applications maintainable and scalable

5.3.2 Libraries in Node.js

Introduction

In Node.js, libraries are pre-written modules or packages that provide specific functionality to simplify application development. These libraries contain reusable code that helps developers avoid writing complex or repetitive logic from scratch. Node.js has access to a vast ecosystem of such libraries through the Node Package Manager (NPM), making it highly efficient and scalable.

Libraries in Node.js are used to handle various operations such as creating servers, connecting to databases, managing authentication, processing HTTP requests, and more. They play a critical role in enhancing productivity and maintaining clean, modular code architecture.

What is a Library in Node.js?

A Node.js library is a set of pre-packaged functions, methods, or classes that serve a specific purpose. These libraries are imported into a Node.js project using either the require() function (CommonJS) or import (ES6 modules).

Libraries can be:

- Built-in (e.g., http, fs, path)
- Third-party (installed using NPM, e.g., express, mongoose, axios)

Types of Node.js Libraries

1. **Web Framework Libraries**
   Used to build web servers and APIs.
   Examples: express, koa, hapi
2. **Database Libraries**
   Enable interaction with databases like MongoDB or MySQL.
   Examples: mongoose, mysql2, sequelize
3. **Utility Libraries**
   Offer helper functions for tasks such as array manipulation, date formatting, etc.
   Examples: lodash, moment, dayjs
4. **Authentication and Security Libraries**
   Provide encryption, token-based login, and password hashing.
   Examples: jsonwebtoken, bcrypt
5. **HTTP Request Libraries**
   Allow the application to communicate with other APIs over the web.
   Examples: axios, node-fetch
6. **File System Libraries**
   Help in file reading, writing, uploading, and directory handling.
   Examples: fs-extra, multer
7. **Environment Configuration Libraries**
   Manage application configuration and sensitive credentials.
   Example: dotenv

Working with a Node.js Library

1. **Installation via NPM:**

   bash

   CopyEdit

   npm install library-name

Example:

bash

CopyEdit

npm install express

2. **Importing the Library:**

   javascript

   CopyEdit

   const express = require('express');

3. **Using the Library: Use available methods or classes to implement desired features.**

   Example using express:

   javascript

   CopyEdit

   const app = express();

   app.get('/', (req, res) => res.send('Hello World'));

   app.listen(3000);

Benefits of Using Libraries in Node.js

- Time-saving: Avoid writing boilerplate or repetitive code.
- Community-tested: Libraries are often well-maintained and secure.
- Modular design: Encourages clean separation of concerns.
- Scalability: Easily extendable with the help of external libraries.
- Improved productivity: Focus more on application logic rather than low-level implementation.

Packaging in Node.js

Introduction

In Node.js, packaging refers to the process of structuring, managing, and distributing a project or module in a standardized format. Node.js uses the Node Package Manager (NPM) to handle these packages, which allows developers to share their code, reuse modules, and manage dependencies effectively. Every Node.js application is treated as a package and is defined by a core file called package.json.

Purpose of Packaging

The main objectives of packaging in Node.js are:

- To organize code and dependencies systematically.
- To manage third-party modules efficiently.
- To provide metadata about the project (such as name, version, author).
- To enable easy distribution and reuse of modules across different applications.
- To support version control and maintain consistency across development environments.

Components of Node.js Packaging

1. **package.json File**

   This is the most important file in any Node.js project. It contains metadata about the project and manages the project's dependencies and scripts.

   Key elements in package.json:

   - name: Name of the package.
   - version: Current version of the package.
   - description: Brief about the project.
   - main: Entry point of the application (e.g., app.js).
   - scripts: Custom commands (e.g., start, test).
   - dependencies: Packages required in production.
   - devDependencies: Packages required only in development.

   Example:

   json

   CopyEdit

```
{
  "name": "myapp",
  "version": "1.0.0",
  "main":        "app.js",
  "scripts": {
    "start": "node app.js"
```

```
  },
  "dependencies":      {
    "express": "^4.18.2"
  }
}
```

2. **package-lock.json File**
    - Automatically generated when dependencies are installed.
    - Records the exact version of each installed package.
    - Ensures consistent installs across environments.
3. **node_modules Directory**
    - Contains all installed libraries and their internal dependencies.
    - Automatically created by running npm install.

Creating a Node.js Package

1. **Initialize the project:**

   bash

   CopyEdit

   npm init

   or

   bash

   CopyEdit

   npm init -y
2. **Install required libraries:**

   bash

   CopyEdit

   npm install express
3. **Run the project using scripts defined in package.json:**

   bash

   CopyEdit

npm start

Publishing a Node.js Package

To share your Node.js package publicly on NPM:

1. Create an NPM account: https://www.npmjs.com
2. Login via terminal:

   bash

   CopyEdit

   npm login
3. Publish the package:

   bash

   CopyEdit

   npm publish

Note: Ensure the package name is unique and the package.json is correctly filled out.

Advantages of Packaging in Node.js

- Efficient management of dependencies.
- Easy reuse and distribution of code.
- Clear project structure and automation via scripts.
- Better version control and reproducibility.
- Integration with NPM ecosystem and community support.

5.3.3 Introduction of MongoDB

MongoDB is a powerful, open-source NoSQL database designed for handling large volumes of data in a flexible and scalable way. Unlike traditional relational databases (like MySQL or Oracle), MongoDB stores data in document-oriented format using BSON (Binary JSON), which makes it highly adaptable to modern application needs.
As a schema-less database, MongoDB allows developers to store data without defining the structure in advance. This flexibility is ideal for applications with dynamic or complex data models—such as crowdfunding platforms—where each record (like a campaign or user profile) might have different sets of attributes.
MongoDB is built for performance, high availability, and horizontal scalability. It supports indexing, replication, and sharding, making it suitable for distributed systems and cloud-based applications.

Why MongoDB is Used as a Database?
In a project like Crowdfunding Using Blockchain, MongoDB is ideal because:
It can easily store campaign details, user information, funding history, etc., without needing a fixed structure.
It allows storing complex, nested data (like transactions within a campaign) in a single document.
It supports fast queries, which is important when showing real-time data to users.

How MongoDB Stores Data?
MongoDB stores data in the following structure:
Database → contains collections
Collection → contains documents
Document → actual data (in BSON format, like JSON)

Example:

```
{
  "campaign_name": "Save the Forests",
  "creator": "John Doe",
  "target_amount": 5000,
  "raised_amount": 3200,
  "backers": [
    { "name": "Alice", "amount": 100 },
    { "name": "Bob", "amount": 200 }
  ]
     }
```

Key Features of MongoDB

1. High Scalability

MongoDB is built with a distributed architecture, allowing it to scale horizontally across multiple servers. This makes it ideal for applications requiring massive data storage and high-speed processing, such as e-commerce platforms, financial services, and social media networks.

2. Hybrid Data Model

Unlike traditional databases that force users to choose between SQL (structured) and NoSQL (unstructured) models, MongoDB supports both. This flexibility enables developers to store structured data (like user profiles) alongside unstructured data (like JSON documents or time-series data) in the same system.

3. Real-Time Data Processing

MongoDB is optimized for real-time analytics and event-driven applications. It supports in-memory caching, stream processing, and low-latency queries, making it suitable for use cases like fraud detection, recommendation engines, and IoT sensor data analysis.

4. Strong Consistency & ACID Compliance

While many NoSQL databases sacrifice consistency for speed, MongoDB ensures ACID (Atomicity, Consistency, Isolation, Durability) compliance, making it reliable for financial transactions, healthcare records, and other critical applications.

5. Built-in AI & Machine Learning Support

MongoDB integrates machine learning capabilities directly into the database, allowing developers to run predictive analytics and AI models without needing external tools. This reduces complexity and speeds up decision-making processes.

CHAPTER 6

CONCLUSION

In conclusion, the integration of blockchain technology into existing crowdfunding platforms presents a groundbreaking opportunity to revolutionize the way funds are raised and managed. By leveraging the decentralized ledger and smart contract capabilities of blockchain, the proposed system ensures enhanced security, transparency, and efficiency throughout the crowdfunding process. The automation of agreements between fundraisers and backers through smart contracts fosters trust and accountability, while the immutable nature of blockchain records provides a tamper-proof record of all transactions. Moreover, the use of cryptocurrency facilitates seamless cross-border transactions, eliminating the barriers associated with traditional banking systems and enhancing accessibility for a global audience. Overall, the proposed system offers a robust and innovative solution to the challenges faced by current crowdfunding platforms. By addressing security vulnerabilities, improving transparency, and streamlining the fundraising process, the system facilitates greater participation and success in fundraising endeavors. As blockchain technology continues to evolve, its potential to reshape the crowdfunding landscape and unlock new opportunities for innovation and collaboration remains promising.

FUTURE ENHANCEMENT

In envisioning future enhancements for the proposed crowdfunding platform leveraging blockchain technology, several avenues for advancement can be explored. Firstly, further integration of advanced cryptographic techniques could be pursued to enhance the security and privacy of user data and transactions on the platform. This may include the adoption of zero-knowledge proofs or homomorphic encryption to enable secure and private computation without revealing sensitive information. Additionally, the implementation of decentralized identity solutions could offer users greater control over their personal data and enhance trust and authenticity within the crowdfunding ecosystem. Decentralized identity platforms built on blockchain can enable individuals to manage and verify their identities securely, reducing the reliance on centralized authorities and mitigating the risk of identity theft or fraud. Moreover, the introduction of decentralized governance mechanisms could empower stakeholders within the crowdfunding community to participate in decision-making processes and contribute to platform

governance. Decentralized autonomous organizations (DAOs) powered by smart contracts can enable transparent and democratic governance, allowing users to vote on platform updates, policy changes, or the allocation of community funds. Furthermore, the integration of artificial intelligence and machine learning algorithms could enable predictive analytics and personalized recommendations to improve campaign success rates and enhance user engagement on the platform. By analyzing historical data and user behavior, AI-powered algorithms can identify trends, predict campaign outcomes, and provide actionable insights to fundraisers and backers. Finally, exploring interoperability with other blockchain networks and decentralized finance (DeFi) platforms could unlock new opportunities for fundraising, liquidity provision, and financial innovation within the crowdfunding ecosystem. Interoperability standards and protocols can enable seamless integration with other blockchain networks, while DeFi protocols can facilitate liquidity pools, tokenization, and decentralized lending for crowdfunding campaigns. Overall, by embracing these future enhancements, the proposed crowdfunding platform can continue to evolve and adapt to meet the changing needs and preferences of its users, ultimately fostering a more inclusive, transparent, and efficient crowdfunding ecosystem powered by blockchain technology.

APPENDICES

APPENDIX A - SOURCE CODE:

Frontend Code:

Components:

Login.jsx.

```
import { useState } from "react";
import axios from "axios";
import { useNavigate } from "react-router-dom";
import { FontAwesomeIcon } from "@fortawesome/react-fontawesome";
import { faEnvelope, faLock, faSignInAlt } from "@fortawesome/free-solid-svg-icons";

const Login = () => {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!email || !password) {
      setError("Please fill in all fields");
      return;
    }

    try {
      const response = await axios.post("http://localhost:5000/api/auth/login", { email, password },
{ withCredentials: true });

      alert("Login successful");
      console.log("Redirecting to dashboard...");

      navigate("/dashboard", { replace: true }); // Redirect to dashboard
    } catch (err) {
```

```
    setError(err.response?.data?.message || "Login failed, please try again.");
  }
 };

 return (
  <div className="min-h-screen bg-gray-900 flex justify-center items-center">
   <div className="bg-gray-800 p-8 rounded-lg shadow-lg max-w-sm w-full">
    <h2 className="text-3xl font-bold text-center text-blue-500 mb-6">Login</h2>

    {error && <p className="text-red-500 text-center mb-4">{error}</p>}

    <form onSubmit={handleSubmit} className="space-y-4">
     <div className="flex items-center mb-4">
      <FontAwesomeIcon icon={faEnvelope} className="text-blue-500 text-2xl mr-4" />
      <input
       type="email"
       value={email}
       onChange={(e) => setEmail(e.target.value)}
       placeholder="Email"
            className="w-full p-3 border border-gray-600 rounded-md focus:outline-none
focus:ring-2 focus:ring-blue-500 bg-gray-700 text-white"
      />
     </div>

     <div className="flex items-center mb-4">
      <FontAwesomeIcon icon={faLock} className="text-red-500 text-2xl mr-4" />
      <input
       type="password"
       value={password}
       onChange={(e) => setPassword(e.target.value)}
       placeholder="Password"
            className="w-full p-3 border border-gray-600 rounded-md focus:outline-none
focus:ring-2 focus:ring-blue-500 bg-gray-700 text-white"
      />
```

```
        </div>

        <button
          type="submit"
          className="w-full py-3 bg-blue-500 text-white rounded-md hover:bg-blue-600 transition flex items-center justify-center gap-2"
        >
          <FontAwesomeIcon icon={faSignInAlt} className="text-white text-lg" />
          Login
        </button>
      </form>

      <div className="mt-4 text-center">
        <p className="text-gray-400">
          Don't have an account?{" "}
          <a href="/register" className="text-blue-500 hover:text-blue-400">
            Register here
          </a>
        </p>
      </div>
    </div>
  </div>
 );
};

export default Login;
```

Register.jsx.

```
import { useState } from "react";
import axios from "axios";
import { useNavigate } from "react-router-dom";
import { FontAwesomeIcon } from '@fortawesome/react-fontawesome';
import { faUser, faEnvelope, faLock, faArrowRight } from '@fortawesome/free-solid-svg-icons';
// Import icons
```

```
const Register = () => {
  const [username, setUsername] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const navigate = useNavigate(); // Initialize useNavigate hook

  const handleSubmit = async (e) => {
    e.preventDefault();

    try {
      await axios.post("http://localhost:5000/api/auth/register", {
        username,
        email,
        password,
      });

      alert("User registered successfully");

      // Redirect to login page after successful registration
      navigate("/login"); // Redirect to the login page
    } catch (error) {
      console.log(error.response?.data);
    }
  };

  return (
    <div className="flex justify-center items-center min-h-screen bg-gray-900">
      <div className="bg-gray-800 p-8 rounded-lg shadow-lg max-w-sm w-full">
        <h2 className="text-3xl font-bold text-center text-blue-500 mb-6">Register</h2>
        <form onSubmit={handleSubmit} className="space-y-6">
          {/* Username Input Field with Icon */}
          <div className="flex items-center space-x-4 mb-4">
            <FontAwesomeIcon icon={faUser} className="text-yellow-400 text-2xl" />
            <input
```

```
          type="text"
          value={username}
          onChange={(e) => setUsername(e.target.value)}
          placeholder="Username"
                 className="w-full p-3 border border-gray-600 rounded-md focus:outline-none
focus:ring-2 focus:ring-blue-500 bg-gray-700 text-white"
         />
        </div>

        {/* Email Input Field with Icon */}
        <div className="flex items-center space-x-4 mb-4">
         <FontAwesomeIcon icon={faEnvelope} className="text-green-400 text-2xl" />
         <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          placeholder="Email"
                 className="w-full p-3 border border-gray-600 rounded-md focus:outline-none
focus:ring-2 focus:ring-blue-500 bg-gray-700 text-white"
         />
        </div>

        {/* Password Input Field with Icon */}
        <div className="flex items-center space-x-4 mb-4">
         <FontAwesomeIcon icon={faLock} className="text-blue-400 text-2xl" />
         <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          placeholder="Password"
                 className="w-full p-3 border border-gray-600 rounded-md focus:outline-none
focus:ring-2 focus:ring-blue-500 bg-gray-700 text-white"
         />
        </div>
```

```
        {/* Register Button with Icon and Hover Effect */}
        <button
          type="submit"
            className="w-full py-3 bg-blue-500 text-white rounded-md flex items-center justify-center space-x-3 hover:bg-blue-600 transition"
        >
          <span>Register</span>
                <FontAwesomeIcon icon={faArrowRight} className="text-white text-lg group-hover:text-gray-300 transition" />
        </button>
       </form>
     </div>
   </div>
  );
};

export default Register;
```

Home.jsx

```
import React from 'react';
import { FontAwesomeIcon } from '@fortawesome/react-fontawesome';
import { faLightbulb, faNewspaper, faUsers, faHandsHelping, faBullhorn, faChartLine } from '@fortawesome/free-solid-svg-icons';
import { faEthereum } from '@fortawesome/free-brands-svg-icons';
import Footer from './Footer';
const Home = () => {
  // Sample data for crowdfunding tips, news, about, and feedbac

  const feedbacks = [
    { name: "John Doe", feedback: "This platform helped me raise funds for my startup!", icon: faUsers, imageUrl: "https://i.pinimg.com/736x/42/47/3e/42473ebf3441c680fa2b82d31cb81e6f.jpg" },
    { name: "Jane Smith", feedback: "A great experience with a supportive community!", icon: faUsers, imageUrl:
```

```
"https://i.pinimg.com/736x/01/00/d1/0100d18d2381d9e968cc1ddfeeb7bc50.jpg" },
  { name: "Alice Johnson", feedback: "I love how easy it is to start a campaign!", icon: faUsers, imageUrl: "https://i.pinimg.com/736x/2b/a3/db/2ba3dbc3473ac3906e0c7ffe42f86f85.jpg" },
 ];

 return (
  <div>
  <div className="min-h-screen bg-gray-900 text-gray-100">
   {/* Navbar */}
   <nav className="bg-gray-800 shadow-lg w-full">
    <div className="max-w-7xl mx-auto px-4">
     <div className="flex items-center justify-between h-16">
      <div className="flex items-center">
       <h1 className="text-3xl font-bold text-purple-400">Startup Singam</h1>
       <FontAwesomeIcon icon={faEthereum} className="text-4xl text-purple-400 ml-4" />
      </div>
      <div className="flex">
       <button className="px-4 py-2 rounded-md text-sm font-medium bg-gray-700 text-white hover:bg-gray-600 focus:outline-none focus:ring-2 focus:ring-purple-500">
        Home
       </button>
       <a href='/about' ><button className="px-4 py-2 rounded-md text-sm font-medium bg-gray-700 text-white hover:bg-gray-600 focus:outline-none focus:ring-2 focus:ring-purple-500 ml-2">
        About
       </button></a>
       <a href='/contact'><button className="px-4 py-2 rounded-md text-sm font-medium bg-gray-700 text-white hover:bg-gray-600 focus:outline-none focus:ring-2 focus:ring-purple-500 ml-2">
        Contact
       </button></a>
       <a href='/dashboard'>
         <button className="px-4 py-2 rounded-md text-sm font-medium bg-purple-600 text-white hover:bg-purple-700 focus:outline-none focus:ring-2 focus:ring-purple-500 ml-2">
          Dashboard
```

```
            </button>
          </a>
        </div>
      </div>
    </div>
  </nav>

  {/* Banner Section */}
  <div className="flex justify-center mb-6">
    <img
      src="https://i.pinimg.com/1200x/14/fd/7e/14fd7e7256275b985ae29d6952ca0073.jpg"  // Replace with your banner image URL
      alt="Crowdfunding Banner"
      className="w-full h-auto rounded-lg"
    />
  </div>



  {/* Feedback Section */}
  <div className="max-w-7xl mx-auto px-4 mb-8">
    <h2 className="text-3xl font-bold text-purple-400 mb-4">User Feedback</h2>
    <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 gap-6">
      {feedbacks.map((feedback, index) => (
        <div key={index} className="bg-gradient-to-r from-red-500 via-pink-500 to-purple-500 rounded-lg p-4 shadow-lg hover:shadow-2xl transition-transform transform hover:scale-105">
          <div className="flex items-center mb-4">
            <FontAwesomeIcon icon={feedback.icon} className="text-3xl text-white mr-4" />
            <h3 className="text-xl font-semibold text-white">{feedback.name}</h3>
          </div>
          <img src={feedback.imageUrl} alt={feedback.name} className="w-full h-40 object-cover rounded-lg mb-4" />
          <p className="text-gray-100">"{feedback.feedback}"</p>
        </div>
```

```
        ))}
       </div>
     </div>
   </div>
   <Footer />
   </div>
  );
};

export default Home;
```

View Project.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import { FontAwesomeIcon } from '@fortawesome/react-fontawesome';
import {
  faEye,
  faHandHoldingUsd,
  faBullseye,
  faCheckCircle,
  faTimes,
  faCalendarAlt,
  faUser,
  faInfoCircle,
  faChartPie,
  faExclamationTriangle
} from '@fortawesome/free-solid-svg-icons';
import { faEthereum } from '@fortawesome/free-brands-svg-icons';

const ViewProjects = () => {
```

```
const [projects, setProjects] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState("");
const [donationAmounts, setDonationAmounts] = useState({});
const [activeDonationId, setActiveDonationId] = useState(null);
const [donationDetails, setDonationDetails] = useState(null);
const [showDonationModal, setShowDonationModal] = useState(false);
const [selectedProject, setSelectedProject] = useState(null);
const [showProjectModal, setShowProjectModal] = useState(false);
const [showStatsModal, setShowStatsModal] = useState(false);
const [ethPrice, setEthPrice] = useState(0);
const [alert, setAlert] = useState({ show: false, message: "", type: "" });

useEffect(() => {
  const fetchProjects = async () => {
    try {
      const response = await axios.get("http://localhost:5000/api/project/view-all-projects", {
        withCredentials: true,
      });

      if (response.status === 200) {
        setProjects(response.data.projects);
        const initialAmounts = {};
        response.data.projects.forEach(project => {
          initialAmounts[project._id] = { usd: "", eth: "" };
        });
        setDonationAmounts(initialAmounts);
      }
    } catch (error) {
      setError(error.response?.data?.message || "Error fetching projects.");
    } finally {
      setLoading(false);
    }
  };
```

```
    const fetchEthPrice = async () => {
      try {
                                                const response = await
axios.get("https://api.coingecko.com/api/v3/simple/price?ids=ethereum&vs_currencies=usd");
        setEthPrice(response.data.ethereum.usd);
      } catch (error) {
        console.error("Error fetching ETH price:", error);
      }
    };

    fetchProjects();
    fetchEthPrice();
  }, []);

  const showAlert = (message, type = "error") => {
    setAlert({ show: true, message, type });
    setTimeout(() => setAlert({ show: false, message: "", type: "" }), 5000);
  };

  const handleViewProject = (project) => {
    setSelectedProject(project);
    setShowProjectModal(true);
  };

  const handleDonate = async (projectId) => {
    const amount = parseFloat(donationAmounts[projectId].usd);
    const project = projects.find(p => p._id === projectId);

    // Check if project is fully funded
    if (project.currentAmount >= project.targetAmount) {
      showAlert("This project is already fully funded!", "error");
      return;
    }
```

```
    // Check if amount is valid
    if (!amount || amount <= 0) {
      showAlert("Please enter a valid donation amount", "error");
      return;
    }

    // Check if donation would exceed the target amount
    const remainingAmount = project.targetAmount - project.currentAmount;
    if (amount > remainingAmount) {
      showAlert(`You can donate maximum ${remainingAmount.toFixed(2)} USD to fully fund this
project`, "warning");
      return;
    }

    try {
      setActiveDonationId(projectId);
      const response = await axios.post(
        `http://localhost:5000/api/project/donate/${projectId}`,
        { amount: Number(amount) },
        { withCredentials: true }
      );

      const updatedProject = response.data.project;
      setProjects(prevProjects =>
        prevProjects.map(project =>
          project._id === updatedProject._id ? updatedProject : project
        )
      );

      setDonationDetails({
        username: response.data.user?.username || "Anonymous",
        amount: amount,
        txHash: response.data.txHash || "0x123...abc",
```

```
      projectTitle: updatedProject.title
    });

    setDonationAmounts(prev => ({ ...prev, [projectId]: { usd: "", eth: "" } }));
    setError("");
    setShowDonationModal(true);
  } catch (error) {
    showAlert(error.response?.data?.message || "Error donating to project", "error");
  } finally {
    setActiveDonationId(null);
  }
};

const handleDonationAmountChange = (projectId, value, type) => {
  const newAmounts = { ...donationAmounts };
  const numericValue = parseFloat(value) || 0;

  if (type === "usd") {
    newAmounts[projectId].usd = value;
    newAmounts[projectId].eth = ethPrice > 0 ? (numericValue / ethPrice).toFixed(6) : "0";
  } else {
    newAmounts[projectId].eth = value;
    newAmounts[projectId].usd = (numericValue * ethPrice).toFixed(2);
  }

  setDonationAmounts(newAmounts);
};

const calculateProgress = (current, target) => {
  return Math.min(Math.round((current / target) * 100), 100);
};

const closeModal = () => {
  setShowDonationModal(false);
```

```
  setDonationDetails(null);
 };

 const closeProjectModal = () => {
  setShowProjectModal(false);
  setSelectedProject(null);
 };

 const closeStatsModal = () => {
  setShowStatsModal(false);
 };

 const totalProjects = projects.length;
 const totalAmountRaised = projects.reduce((acc, project) => acc + (project.currentAmount || 0),
0);

 if (loading) {
  return (
   <div className="flex justify-center items-center min-h-screen bg-gray-900">
    <div className="text-xl text-purple-400 animate-pulse">Loading Projects...</div>
   </div>
  );
 }

 if (error) {
  return (
   <div className="flex justify-center items-center min-h-screen bg-gray-900">
    <div className="text-xl text-red-400">{error}</div>
   </div>
  );
 }

 return (
  <div className="min-h-screen bg-gray-900 py-8 px-4 sm:px-6 lg:px-8">
```

```
    {/* Alert Notification */}
    {alert.show && (
      <div className={`fixed top-4 right-4 z-50 p-4 rounded-md shadow-lg ${
        alert.type === "error" ? "bg-red-600" : "bg-yellow-600"
      } text-white max-w-md`}>
        <div className="flex items-center">
          <FontAwesomeIcon
            icon={alert.type === "error" ? faTimes : faExclamationTriangle}
            className="mr-2"
          />
          <span>{alert.message}</span>
        </div>
      </div>
    )}

    {/* Stats Modal */}
    {showStatsModal && (
      <div className="fixed inset-0 bg-black bg-opacity-75 flex items-center justify-center z-50
p-4">
        <div className="bg-gray-800 rounded-lg max-w-md w-full p-6 relative">
          <button
            onClick={closeStatsModal}
            className="absolute top-4 right-4 text-gray-400 hover:text-white"
          >
            <FontAwesomeIcon icon={faTimes} size="lg" />
          </button>

          <h3 className="text-2xl font-bold text-white mb-4">Project Statistics</h3>
          <div className="space-y-4">
            <div className="flex justify-between">
              <span className="text-gray-400">Total Projects:</span>
              <span className="text-white font-medium">{totalProjects}</span>
            </div>
            <div className="flex justify-between">
```

```
            <span className="text-gray-400">Total Amount Raised:</span>
                    <span  className="text-white  font-medium">{totalAmountRaised.toFixed(2)}
USD</span>
           </div>
          </div>
         </div>
       </div>
     )}

     {/* Donation Success Modal */}
     {showDonationModal && donationDetails && (
       <div className="fixed inset-0 bg-black bg-opacity-75 flex items-center justify-center z-50
p-4">
         <div className="bg-gray-800 rounded-lg max-w-md w-full p-6 relative">
          <button
           onClick={closeModal}
           className="absolute top-4 right-4 text-gray-400 hover:text-white"
          >
           <FontAwesomeIcon icon={faTimes} size="lg" />
          </button>

          <div className="text-center mb-6">
           <FontAwesomeIcon
            icon={faCheckCircle}
            className="text-green-500 text-5xl mb-4"
           />
           <h3 className="text-2xl font-bold text-white mb-2">Donation Successful!</h3>
           <p className="text-gray-300">Thank you for your support</p>
          </div>

          <div className="space-y-4">
           <div className="flex justify-between">
            <span className="text-gray-400">Project:</span>
            <span className="text-white font-medium">{donationDetails.projectTitle}</span>
```

```
            </div>

            <div className="flex justify-between">
              <span className="text-gray-400">Donor:</span>
              <span className="text-white font-medium">{donationDetails.username}</span>
            </div>

            <div className="flex justify-between">
              <span className="text-gray-400">Amount:</span>
              <span className="text-white font-medium flex items-center">
                <FontAwesomeIcon icon={faEthereum} className="mr-1" />
                {donationDetails.amount} USD
              </span>
            </div>

            <div className="flex justify-between items-center">
              <span className="text-gray-400">Transaction:</span>
              <span className="text-purple-400 font-mono text-sm truncate max-w-[180px]">
                {donationDetails.txHash}
              </span>
            </div>
          </div>

          <button
            onClick={closeModal}
            className="w-full mt-6 py-2 bg-purple-600 hover:bg-purple-700 text-white rounded-md
transition"
          >
            Close
          </button>
        </div>
      </div>
    )}
```

```
    {/* Project Details Modal */}
    {showProjectModal && selectedProject && (
      <div className="fixed inset-0 bg-black bg-opacity-75 flex items-center justify-center z-50
p-4">
        <div className="bg-gray-800 rounded-lg max-w-2xl w-full p-6 relative">
          <button
            onClick={closeProjectModal}
            className="absolute top-4 right-4 text-gray-400 hover:text-white"
          >
            <FontAwesomeIcon icon={faTimes} size="lg" />
          </button>

          <div className="flex flex-col md:flex-row gap-6">
            {selectedProject.imageUrl && (
              <div className="md:w-1/2">
                <img
                  src={selectedProject.imageUrl}
                  alt={selectedProject.title}
                  className="w-full h-64 object-cover rounded-lg"
                />
              </div>
            )}

            <div className="md:w-1/2">
              <h2 className="text-2xl font-bold text-white mb-4">{selectedProject.title}</h2>

              <div className="space-y-4">
                <div className="flex items-center text-gray-300">
                  <FontAwesomeIcon icon={faUser} className="mr-2 w-4" />
                  <span>Created by: {selectedProject.creator?.username || "Unknown"}</span>
                </div>

                <div className="flex items-center text-gray-300">
                  <FontAwesomeIcon icon={faCalendarAlt} className="mr-2 w-4" />
```

Created on: {new Date(selectedProject.createdAt).toLocaleDateString()}

<h3 className="text-lg font-semibold text-white mb-2 flex items-center">
<FontAwesomeIcon icon={faInfoCircle} className="mr-2" />
Description
</h3>
{selectedProject.description}

<FontAwesomeIcon icon={faEthereum} className="mr-1" />
{selectedProject.currentAmount || 0} raised

<FontAwesomeIcon icon={faEthereum} className="mr-1" />
{selectedProject.targetAmount} goal

<div
className={`h-2.5 rounded-full ${calculateProgress(selectedProject.currentAmount || 0, selectedProject.targetAmount) >= 100 ? 'bg-green-500' : 'bg-purple-500'}`}
style={{ width: `${calculateProgress(selectedProject.currentAmount || 0, selectedProject.targetAmount)}%` }}
>

{calculateProgress(selectedProject.currentAmount || 0, selectedProject.targetAmount)}% funded

```
            </div>
          </div>

          <div className="pt-4">
            <div className="flex flex-col space-y-4">
              <div className="relative">
                <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none">
                <span className="text-gray-400">$</span>
                </div>
                <input
                type="number"
                min="1"
                max={selectedProject.targetAmount - (selectedProject.currentAmount || 0)}
                className="w-full pl-8 px-4 py-2 bg-gray-700 border border-gray-600 rounded-md text-white focus:ring-2 focus:ring-purple-500 focus:border-transparent"
                placeholder="Donation amount (USD)"
                value={donationAmounts[selectedProject._id]?.usd || ""}
                onChange={(e) => handleDonationAmountChange(selectedProject._id, e.target.value, "usd")}
                />
              </div>
              <div className="relative">
                <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none">
                <FontAwesomeIcon icon={faEthereum} className="text-gray-400" />
                </div>
                <input
                type="number"
                min="0"
                max={(selectedProject.targetAmount - (selectedProject.currentAmount || 0)) / ethPrice}
                step="0.000001"
                className="w-full pl-8 px-4 py-2 bg-gray-700 border border-gray-600 rounded-md text-white focus:ring-2 focus:ring-purple-500 focus:border-transparent"
```

```
                placeholder="Donation amount (ETH)"
                value={donationAmounts[selectedProject._id]?.eth || ""}
                        onChange={(e) => handleDonationAmountChange(selectedProject._id,
e.target.value, "eth")}
                />
              </div>
            </div>

            <button
              onClick={() => handleDonate(selectedProject._id)}
            disabled={activeDonationId === selectedProject._id || selectedProject.currentAmount
>= selectedProject.targetAmount}
              className={`w-full mt-2 py-2 text-white rounded-md transition flex items-center
justify-center ${
                activeDonationId === selectedProject._id || selectedProject.currentAmount >=
selectedProject.targetAmount
                ? 'bg-purple-800 cursor-not-allowed'
                : 'bg-purple-600 hover:bg-purple-700'
              }`}
            >
              {activeDonationId === selectedProject._id ? (
                'Processing...'
              ) : selectedProject.currentAmount >= selectedProject.targetAmount ? (
                'Project Fully Funded'
              ) : (
                <>
                  <FontAwesomeIcon icon={faHandHoldingUsd} className="mr-2" />
                  Donate Now
                </>
              )}
            </button>
          </div>
        </div>
      </div>
    </div>
```

```
        </div>
      </div>
    )}

    <div className="max-w-7xl mx-auto">
      <div className="flex justify-between items-center mb-8">
        <h2 className="text-3xl font-bold text-purple-400">
          <FontAwesomeIcon icon={faBullseye} className="mr-2" />
          Explore Projects
        </h2>
        <button
          onClick={() => setShowStatsModal(true)}
          className="py-2 px-4 bg-blue-600 hover:bg-blue-700 text-white rounded-md flex items-
center"
        >
          <FontAwesomeIcon icon={faChartPie} className="mr-2" />
          View Stats
        </button>
      </div>

      {projects.length === 0 && (
        <div className="text-center text-gray-400 py-12">
          No projects available yet. Be the first to create one!
        </div>
      )}

      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
        {projects.map((project) => {
          const progress = calculateProgress(project.currentAmount || 0, project.targetAmount);
          const isFunded = progress >= 100;
          const remainingAmount = project.targetAmount - (project.currentAmount || 0);

          return (
            <div key={project._id} className="bg-gray-800 rounded-lg shadow-lg hover:shadow-
```

```
xl transition-all duration-300 overflow-hidden">
            {project.imageUrl && (
              <img
                src={project.imageUrl}
                alt={project.title}
                className="w-full h-48 object-cover"
              />
            )}

            <div className="p-6">
              <h3 className="text-xl font-semibold text-white mb-2">{project.title}</h3>
              <p className="text-gray-400 mb-4 line-clamp-3">{project.description}</p>

              <div className="mb-4">
                <div className="flex justify-between text-sm text-gray-300 mb-1">
                  <span>
                    <FontAwesomeIcon icon={faEthereum} className="mr-1" />
                    {project.currentAmount || 0} raised
                  </span>
                  <span>
                    <FontAwesomeIcon icon={faEthereum} className="mr-1" />
                    {project.targetAmount} goal
                  </span>
                </div>
                <div className="w-full bg-gray-700 rounded-full h-2.5">
                  <div
                    className={`h-2.5 rounded-full ${isFunded ? 'bg-green-500' : 'bg-purple-500'}`}
                    style={{ width: `${progress}%` }}
                  ></div>
                </div>
                <div className="text-right mt-1 text-xs text-gray-400">
                  {progress}% funded
                </div>
              </div>
```

```
            <div className="flex justify-between text-gray-300 text-sm mb-4">
              <div>
                <FontAwesomeIcon icon={faHandHoldingUsd} className="mr-1" />
                {project.donations?.length || 0} donations
              </div>
              {isFunded && (
                <div className="text-green-400">
                  <FontAwesomeIcon icon={faCheckCircle} className="mr-1" />
                  Funded!
                </div>
              )}
            </div>

            <div className="space-y-3">
              <button
                onClick={() => handleViewProject(project)}
                  className="w-full py-2 bg-blue-600 hover:bg-blue-700 text-white rounded-md
transition flex items-center justify-center"
              >
                <FontAwesomeIcon icon={faEye} className="mr-2" />
                View Details
              </button>

              {!isFunded && (
                <div className="space-y-2">
                  <div className="relative">
                    <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-
none">
                      <span className="text-gray-400">$</span>
                    </div>
                    <input
                      type="number"
                      min="1"
```

```
                  max={remainingAmount}
                 className="w-full pl-8 px-4 py-2 bg-gray-700 border border-gray-600 rounded-
md text-white focus:ring-2 focus:ring-purple-500 focus:border-transparent"
                  placeholder={`Max ${remainingAmount.toFixed(2)} USD`}
                  value={donationAmounts[project._id]?.usd || ""}
                    onChange={(e) => handleDonationAmountChange(project._id, e.target.value,
"usd")}
                />
               </div>
               <div className="relative">
                 <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-
none">
                  <FontAwesomeIcon icon={faEthereum} className="text-gray-400" />
                 </div>
                 <input
                  type="number"
                  min="0"
                  max={remainingAmount / ethPrice}
                  step="0.000001"
                 className="w-full pl-8 px-4 py-2 bg-gray-700 border border-gray-600 rounded-
md text-white focus:ring-2 focus:ring-purple-500 focus:border-transparent"
                  placeholder={`Max ${(remainingAmount / ethPrice).toFixed(6)} ETH`}
                  value={donationAmounts[project._id]?.eth || ""}
                    onChange={(e) => handleDonationAmountChange(project._id, e.target.value,
"eth")}
                />
               </div>

               <button
                onClick={() => handleDonate(project._id)}
                disabled={activeDonationId === project._id || isFunded}
                    className={`w-full py-2 text-white rounded-md transition flex items-center
justify-center ${
                  activeDonationId === project._id || isFunded
                   ? 'bg-purple-800 cursor-not-allowed'
```

```
                    : 'bg-purple-600 hover:bg-purple-700'
                }`}
              >
                {activeDonationId === project._id ? (
                 'Processing...'
                ) : (
                  <>
                    <FontAwesomeIcon icon={faHandHoldingUsd} className="mr-2" />
                    Donate Now
                  </>
                )}
              </button>
            </div>
          )}
        </div>
      </div>
    );
  })}
 </div>
 </div>
 </div>
);
};

export default ViewProjects;
```

Main.jsx:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import 'bootstrap/dist/css/bootstrap.min.css';

createRoot(document.getElementById('root')).render(
```

```
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Index.css:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

App.jsx:

```
#root {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}

.logo {
  height: 6em;
  padding: 1.5em;
  will-change: filter;
  transition: filter 300ms;
}
.logo:hover {
  filter: drop-shadow(0 0 2em #646cffaa);
}
.logo.react:hover {
  filter: drop-shadow(0 0 2em #61dafbaa);
}

@keyframes logo-spin {
  from {
    transform: rotate(0deg);
```

```
  }
  to {
    transform: rotate(360deg);
  }
}

@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}

.card {
  padding: 2em;
}

.read-the-docs {
  color: #888;
}
```

App.jsx

```
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
import Register from "./components/Register";
import Login from "./components/Login";
import Dashboard from "./components/Dashboard";
import AddProject from "./components/AddProject";
import ViewProject from "./components/ViewProject";
import Home from "./components/Home";
import About from "./components/About";
import Contact from "./components/Contact";

function App() {
  return (
    <Router>
      <Routes>
```

```
        <Route path="/register" element={<Register />} />
        <Route path="/login" element={<Login />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/add-project" element={<AddProject />} />
        <Route path="/view-projects" element={<ViewProject />} />
        <Route path="/home" element={<Home />} />
        <Route path="/" element={<Login />} />
        <Route path="/about" element={<About/>}/>
        <Route path="/contact" element={<Contact/>}/>
      </Routes>
    </Router>
  );
}

export default App;
```

BACKEND SOURCE CODE:

```
const express = require("express");
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
const cors = require("cors");
const dotenv = require("dotenv");
const session = require("express-session");
const MongoStore = require("connect-mongo");

dotenv.config();

const app = express();
```

```
// Middleware
app.use(cors({ origin: "http://localhost:5173", credentials: true })); // Allow CORS for the React front-end
app.use(express.json());

// Session middleware
app.use(
  session({
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    cookie: { secure: false, httpOnly: true, maxAge: 1000 * 60 * 60 * 24 }, // 1 day cookie
     store: MongoStore.create({ mongoUrl: process.env.MONGO_URI }), // Store sessions in MongoDB
  })
);

// MongoDB connection (updated to remove deprecated options)
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB connected"))
  .catch((err) => console.log(err));

// Routes
app.use("/api/auth", require("./routes/auth"));
app.use("/api/project", require("./routes/project"));  // Add the project routes

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

REFERENCES

[1] V. Patil, V. Gupta and R. Sarode, "Blockchain- Based Crowdfunding Application", 2021 Fifth International Conference on I-SMAC (IoT inSocial, Mobile, Analytics and Cloud) (I-SMAC),2021, pp. 1546-1553.

[2] Vikas Hassija, Vinay Chamola and Sherali Zeadally, "BitFund: A blockchain-based crowd funding platform for future smart and connected nation", Sustainable Cities and Society, vol. 60, pp. 102145, 2020.

[3] S. Rashmitha, H. A. Sanjay, K. A. Shastry and K.Jayaa Shree Laxmi, "FarmFund - A Blockchain based Crowdfunding App for Farmers", 2022 7th International Conference on Communication and Electronics Systems (ICCES), 2022, pp. 682-689.

[4] Kumari, S., & Parmar, K. (2021). “Secure and Decentralized Crowdfunding Mechanism Based on Blockchain Technology”. In Proceedings of the International Conference on Paradigms of Computing, Communication and Data Sciences (pp. 79–90). Springer Singapore.

[5] Ashari, F. (2020). “Smart Contract and Blockchain for Crowdfunding Platform”. In International Journal of Advanced Trends in Computer Science and Engineering (Vol. 9, Issue3, pp. 3036–3041). The World Academy of Research in Science and Engineering

[6] A. Malve, S. Barhate, S. Sharma "TRUSTED CROWDFUNDING USING SMART CONTRACT", International Journal of Emerging Technologies and Innovative Research (www.jetir.org | UGC and issn Approved), ISSN:2349-5162, Vol.9, Issue 6, page no. pp371- 375, June-2022

[7] R. Gupta, M. Yadav,U.Dhankar, “Crowdfunding using Ethereum Blockchain”, International Journal for Research in Applied Science & Engineering Technology (IJRASET) ISSN: 2321-9653; IC Value: 45.98; SJ Impact Factor: 7.538, Volume 10 Issue V May 2022.

[8] Duan, L., Sun, Y., Zhang, K., & Ding, Y. (2022). "Multiple-Layer Security Threats on the Ethereum Blockchain and Their Countermeasures". Security and Communication Networks, 2022, 5307697, vol. 2022, February 2022.

[9] G. Habib, S. Sharma , S. Ibrahim , I. Ahmad, S. Qureshi, M. Ishfaq, "Blockchain Technology: Benefits, Challenges, Applications, and Integration of Blockchain Technology with Cloud Computing", Department of Computer Science and Engineering, National Institute of Technology, Srinagar 190001, India, Vol.14, Issue 11, November-2022.

[10] H. Taherdoost, "Smart Contracts in Blockchain Technology: A Critical Review”, Department of Arts, Communications and Social Sciences, University Canada West, Vancouver, BC V6B, Canada, Vol.14, Issue 2, February- 2023.